

University of West Attica
School of Engineering
Department of Informatics and Computer Engineering
Digital Systems Design Laboratory

Lab Exercise 9: Designing Sequential Circuits and Register Files

For this exercise, you will upload to eclass, for each question: 1. The circuit code 2. The testbench code 3. Screenshot from the running of the unit (waveform)

1. Design a simple 4-bit register. The register will consist of flip flops type D and will have the following interface and structure

```
LIBRARY ieee; USE
ieee.std_logic_1164.all; ENTITY reg8 IS PORT
( IN STD_LOGIC_VECTOR(3
  D:                                DOWNT0 0);
  Resetn, Clock: IN STD_LOGIC;
  Q:                                OUT STD_LOGIC_VECTOR(3 DOWNT0 0));
END reg8;

ARCHITECTURE behavioral OF reg8 IS
START
  PROCESS (Resetn, Clock)
  START
    IF Resetn = '0' THEN
      Q <= "0000" ;
    ELSIF rising_edge(Clock) THEN
      Q <= D;
    ENDIF;
  END PROCESS;
END behavioral;
```

2. Test the above unit by setting the following as inputs and observing the exits.

In	Out
0010	
1110	
1010	

3. The following VHDL code creates a register file of 4 registers, each of which which consists of 4 bits.

```

ieee library? use
ieee.std_logic_1164.all; USE ieee.std_logic_unsigned.all;
USE ieee.numeric_std.ALL; entity regfile is generic ( dw

                                : natural := 4; size : natural :=
4; addrw : natural := 2); port ( A : in
std_logic_vector(dw-1 downto 0);

                                Addr : in std_logic_vector(addrw-1 downto 0);
                                we      : in std_logic; clk : in std_logic;

                                C : out std_logic_vector(dw-1 downto 0));
end regfile;

architecture behavioral of regfile is

type regArray is array(0 to size-1) of std_logic_vector(dw-1 downto 0); signal regfile : regArray;

begin
process(clk) begin if
(clk'event
and clk='0') then if we='1' then

                                regfile(to_integer(unsigned(Addr))) <= A; endif? endif?

                                C <= regfile(to_integer(unsigned(Addr))); end process?

```

Try its operation as follows:

Write the register to address 00: 0101

Registering the register at address 01: 1101

Register entry at address 10: 0010

Register entry at address 11: 1001

Reading the register at address 01

Reading the register at address 11

4. Extend the register file of the previous exercise by adding two additional ports, one read and one write, as well as an additional data input. The new definition of entity will be the following:

```

ieee library? use
ieee.std_logic_1164.all; USE ieee.std_logic_unsigned.all;
USE ieee.numeric_std.ALL; entity regfile is

```

```
generic ( dw          : natural := 4; size :  
          natural := 4; addrw : natural := 2);  
  
port ( A : in std_logic_vector(dw-1 downto 0); rAddr1: in std_logic_vector(addrw-1 downto  
          0); rAddr2: in std_logic_vector(addrw-1 downto 0); wAddr : in std_logic_vector(addrw-1  
          downto 0);  
  
       we          : in std_logic; clk : in  
       std_logic; reset : in std_logic; : out  
       std_logic_vector(dw-1 downto 0); : out  
       B          std_logic_vector(dw-1 downto 0));  
       C  
  
end regfile;
```

Test the circuit operation as follows:

1. Write the content 0101 to address 00
2. Write the content 1010 to address 01
3. Write the content 0000 to address 10
4. Write the content 1111 to address 11
5. Reading addresses 00 and 10
6. Reading addresses 01 and 11